IAMMAI Protocol

1 - Frame, Purpose, Scope, Invariants, Canonical Architecture

1. Document Status

IAMMAI is a public protocol (framework) which defines the constitutional core by which candidate material may become standing state without semantic collapse, silent rewrite, lineage loss, or illegitimate transition.

This document is normative and declares what must remain true if a later implementation, profile, or adaptation is still to count as IAMMAI-compatible.

This document is not:

- a product manual,
- a software specification,
- a deployment guide,
- a business model,
- a legal opinion,
- or a complete theory of all adjacent systems.

It is the constitutional protocol.

---

2. Foundational Posture

IAMMAI is a public framework for lawful semantic order.

It does not attempt to author reality.

It does not attempt to define meaning.

It does not attempt to own what is prior to the system.

It defines how a system may lawfully relate:

- candidate material,
- standing state,
- lineage,
- authority,
- contribution,
- and consequence

without collapse between them.

---

3. Purpose

IAMMAI exists to preserve coherence between what appears, what may count, what may stand, what may be acted upon, and what follows.

Its purpose is to prevent systems from collapsing:

- signal into truth,
- truth into decision,
- decision into consequence,
- correction into overwrite,
- authority into authorship,
- and interpretation into standing state.

IAMMAI exists so that a system may move from candidate material into standing state with:

- clarity,

- boundedness,
- traceability,
- lawful transition,
- and preserved continuity.

A concise statement of purpose is:

IAMMAI is a public protocol for keeping what happened, what stands, what was decided, and what follows clear, traceable, and impossible to silently rewrite.

---

4. Scope

This core protocol covers:

- canonical phase order,
- distinction between preparation and standing state,
- containment of irreversible exit,
- lawful resolution,
- lineage-preserving succession,
- governance and authority bounds,
- contribution-role distinction,
- minimal representational requirements,
- and conformance discipline.

---

5. Reader Orientation

IAMMAI should be read as lawful architecture, not as motivational philosophy.

It is written to:

- separate,
- order,
- bound,
- and preserve.

---

6. Design Goals

IAMMAI is designed to achieve the following:

6.1 Phase clarity

Each phase must own a distinct semantic job.

6.2 Transition integrity

No standing state may arise through silent short-circuit.

6.3 Lineage preservation

No lawful correction may erase the fact that prior standing state existed.

6.4 Bounded authority

No actor may acquire protocol force merely through access, visibility, or convenience.

6.5 Contribution distinction

Where multiple actors or systems participate, semantic roles must remain distinguishable.

6.6 Representational recoverability

Implementation must preserve enough structure that semantic distinctions remain reconstructable.

6.7 Conformance across variation

Different implementations may vary in expression, but not in foundational semantic identity.

---

7. Non-Goals

IAMMAI does not attempt here to solve:

- all forms of human meaning,
- all forms of dynamic adaptation,
- all future implementation logic,
- all governance constitutions in every domain,
- or all possible kinds of intelligence.

This document defines the constitutional center only.

---

8. Core Invariants

The following invariants govern the whole framework.

8.1 Fixed canonical order

The semantic chain must remain ordered. No later phase may silently impersonate an earlier one.

8.2 No phase collapse

Each phase must remain distinguishable in function and meaning.

8.3 No dark standing

Standing state must not appear without lawful transition.

8.4 No dark consequence

Consequence must not appear as though it authorizes itself retroactively.

8.5 No silent overwrite

Correction must not replace prior state in place as though the prior state never stood.

8.6 No hidden fourth outcome

Lawful resolution is closed. Containment, ambiguity, delay, or disappearance must not masquerade as extra resolution paths.

8.7 No authority-source collapse

Authority to act is not the same as semantic source.

8.8 No contribution-role collapse

Origin, formalization, authorization, rendering, and transmission must not be silently merged where materially distinct.

8.9 Protocol before convenience

No implementation shortcut may override the core semantic structure.

---

9. Canonical Architecture

IAMMAI defines one canonical semantic chain:

signal → acceptance → scope → presence → threshold → truth → continuity → decision → consequence

This chain is divided into two regimes:

9.1 Preparation regime

- signal
- acceptance
- scope
- presence
- threshold

These phases qualify candidate material without yet creating standing state.

9.2 Standing-state regime

- truth
- continuity
- decision
- consequence

These phases form, preserve, commit, and apply standing state.

This split is mandatory.

Preparation is not standing.

Standing is not mere preparation extended.

---

10. Orthogonal Controls

Two protocol elements are not phases in the canonical chain, but remain fundamental.

10.1 Containment

HOLD governs whether an otherwise eligible irreversible exit may proceed.

HOLD is:

- cross-phase,
- orthogonal,
- and non-semantic in kind.

HOLD is not:

- a canonical phase,

- a standing state,
- or a lawful resolution outcome.

10.2 Resolution

Lawful resolution is limited to:

- Finalize
- Invalidate
- Evolve

No fourth outcome is allowed by stealth.

---

11. Foundational Distinctions

The following distinctions are non-negotiable:

- signal is not truth
- admissibility is not standing
- threshold is not truth
- truth is not continuity
- continuity is not decision
- decision is not consequence
- interpretation is not standing
- authority is not authorship
- correction is not overwrite
- effect is not proof of upstream validity

These distinctions are not rhetorical.

They are the anti-collapse spine of the protocol.

---

12. Minimal Formal Boundary

The core architecture may be minimally bounded as follows:

signal(x) → candidate(x)
accepted(x) → signal(x)
scoped(x) → accepted(x)
present(x) → scoped(x)
threshold_met(x) → present(x) ∧ scoped(x)

truth(x) → standing(x)
continuity(x) → standing(x) ∧ traceable(x)
decision(x) → references_truth(x) ∨ references_continuity(x)
consequence(x) → decision(x)

held(x) → ¬resolvable(x)

resolution(x) ∈ {FINALIZE, INVALIDATE, EVOLVE}

These expressions do not replace doctrine.

They bound it enough to prevent casual semantic slippage.

---

13. What This Protocol Is

IAMMAI is the law of standing, not the whole world around standing.

It defines:

- what may count,
- what may stand,
- how it may remain,

- who may act,
- how change stays lawful,
- and how the architecture protects itself from silent collapse.

This is the constitutional center.

14. Canonical Conclusion

IAMMAI exists so that candidate material, standing state, lineage, authority, contribution, and consequence may proceed lawfully without semantic collapse.

Anything outside that purpose lies outside this protocol.

2 - Preparation Regime and Standing-State Regime

15. Preparation Regime

15.1 Purpose

The Preparation Regime governs how candidate material becomes eligible for standing-state transition without itself creating standing state.

Its function is to prevent the first great collapse of systems:

mistaking what is available, admitted, or accumulating for what already stands.

The Preparation Regime consists of:

- signal
- acceptance
- scope
- presence
- threshold

These phases are cumulative in direction, but none of them alone or together constitute standing truth.

---

16. Signal

16.1 Definition

Signal is candidate material made available to the system for possible handling.

Signal is the first protocol-relevant appearance of material.

It may arise from:

- observation,
- declaration,
- submission,
- detection,
- or other forms of system-available occurrence.

16.2 Semantic job

Signal answers:

What is now available to the system as candidate material?

That is all.

Signal does not answer:

- whether it counts,
- whether it is admissible,
- whether it belongs here,
- whether it is true,

- whether it will matter later.

16.3 What signal is not

Signal is not:

- admissibility
- relevance
- proof
- truth
- value
- standing

A system may receive signal and later reject it entirely.

16.4 Minimal formal boundary

signal(x) → candidate(x)
signal(x) ↛ truth(x)
signal(x) ↛ standing(x)
signal(x) ↛ consequence(x)

17. Acceptance

17.1 Definition

Acceptance is the phase by which candidate signal becomes admitted for structured protocol handling.

Acceptance marks the difference between:

- mere appearance,

  and

- lawful intake.

17.2 Semantic job

Acceptance answers:

May this candidate material enter protocol handling?

Acceptance therefore owns admissible intake, not truth.

17.3 What acceptance is not

Acceptance is not:

- scope
- threshold
- truth
- standing

Accepted material may still be:

- irrelevant,
- out of scope,
- insufficient,
- or later invalid.

17.4 Minimal formal boundary

$\text{accepted}(x) \rightarrow \text{signal}(x)$
$\neg\text{accepted}(x) \rightarrow \neg\text{processable}(x)$
$\text{accepted}(x) \nrightarrow \text{truth}(x)$
$\text{accepted}(x) \nrightarrow \text{standing}(x)$

18. Scope

18.1 Definition

Scope is the phase by which accepted material is bound to the matter within which it may lawfully count.

Scope determines bounded applicability.

18.2 Semantic job

Scope answers:

For what matter does this accepted material count?

Without scope, accepted material can bleed across matters and contexts improperly.

18.3 What scope is not

Scope is not:

- presence
- threshold
- truth
- decision

A thing may be in scope without being sufficient, standing, or consequential.

18.4 Minimal formal boundary

$\text{scoped}(x) \rightarrow \text{accepted}(x) \wedge \exists m\ \text{matter_ref}(x, m)$
$\text{scoped}(x) \nrightarrow \text{truth}(x)$
$\text{scoped}(x) \nrightarrow \text{standing}(x)$
$\text{scoped}(x) \nrightarrow \text{consequence}(x)$

19. Presence

19.1 Definition

Presence is the phase in which scoped material becomes relevant occurrence within the matter in a way that may count toward threshold.

Presence is stronger than scope, but weaker than threshold.

19.2 Semantic job

Presence answers:

What relevant occurrence is now present within this matter?

Presence may be singular or cumulative depending on later domain profiles, but at protocol level it remains pre-threshold and pre-standing.

19.3 What presence is not

Presence is not:

- threshold satisfaction
- truth
- standing state
- decision

19.4 Minimal formal boundary

present(x) → scoped(x)
present(x) ↛ threshold_met(x)
present(x) ↛ truth(x)
present(x) ↛ standing(x)

20. Threshold

20.1 Definition

Threshold is the phase at which preparatory conditions become sufficient for standing-state transition to become eligible.

Threshold does not create standing truth.

Threshold creates **eligibility for truth formation**.

20.2 Semantic job

Threshold answers:

Has enough occurred, within lawful scope, for standing truth to become eligible?

This is the boundary phase between preparation and standing-state formation.

20.3 What threshold is not

Threshold is not:

- truth
- continuity
- decision
- consequence

Threshold means readiness for lawful transition, not completion of transition.

20.4 Threshold and irreversible exit

Threshold is the final phase of preparation and the immediate predecessor to standing truth.

The transition:

threshold → truth

is one of the protocol's key irreversible transitions.

That transition may still be prevented by containment.

20.5 Minimal formal boundary

threshold_met(x) → present(x) ∧ scoped(x)
threshold_met(x) → truth_eligible(x)
threshold_met(x) ↛ truth(x)
threshold_met(x) ∧ held(x) → ¬truth_transition_permitted(x)

21. Preparation Regime Summary

The Preparation Regime defines how candidate material becomes eligible for standing-state transition without itself becoming standing state.

Its canonical chain is:

signal → acceptance → scope → presence → threshold

Within this chain:

- signal makes candidate material available,
- acceptance admits it for handling,
- scope binds it to a matter,
- presence recognizes relevant occurrence,
- threshold marks sufficient readiness for standing truth to become eligible.

The entire Preparation Regime exists to prevent premature standing.

---

22. Standing-State Regime

22.1 Purpose

The Standing-State Regime defines how a system forms, preserves, commits, and applies standing state.

Its function is to prevent the second great collapse of systems:

mistaking truth, continuity, decision, and consequence as though they were one event.

The Standing-State Regime consists of:

- truth
- continuity
- decision
- consequence

These phases are irreversible in protocol kind and must remain distinguishable.

---

23. Truth

23.1 Definition

Truth is the phase in which an eligible matter becomes an explicit standing claim within the protocol.

Truth is the first standing-state phase.

23.2 Semantic job

Truth answers:

What now stands as a claim within the protocol?

Truth is not total metaphysical truth.

It is standing truth in protocol form.

23.3 What truth is not

Truth is not:

- threshold
- continuity
- decision
- consequence

Truth forms standing claim.

It does not yet preserve, commit, or apply it.

23.4 Minimal formal boundary

truth(x) → standing_claim(x)
truth(x) → standing(x)
truth(x) ↛ continuity(x)
truth(x) ↛ decision(x)
truth(x) ↛ consequence(x)

24. Continuity

24.1 Definition

Continuity is the phase in which standing truth becomes preserved, lineage-bearing state.

Continuity is not mere duration.

It is preserved standing with traceable relation across time.

24.2 Semantic job

Continuity answers:

What standing truth is now preserved as lineage-bearing state?

24.3 What continuity is not

Continuity is not:

- truth formation
- decision
- consequence
- raw persistence

Continuity preserves standing truth in protocol form.

It does not choose or execute.

24.4 Minimal formal boundary

$\text{continuity}(x) \rightarrow \text{standing}(x) \wedge \text{traceable}(x)$
$\text{continuity}(x) \rightarrow \text{truth}(x) \vee \text{references_truth}(x)$
$\text{continuity}(x) \nrightarrow \text{decision}(x)$
$\text{continuity}(x) \nrightarrow \text{consequence}(x)$

25. Decision

25.1 Definition

Decision is the phase in which continuity-bearing standing state becomes the basis for committed direction.

Decision is not mere evaluation.

It is commitment.

25.2 Semantic job

Decision answers:

What direction now stands committed on the basis of preserved state?

25.3 What decision is not

Decision is not:

- truth
- continuity
- consequence
- effect itself

Decision commits direction.

It does not yet apply the result.

25.4 Minimal formal boundary

$\text{decision}(x) \rightarrow \text{commitment}(x)$
$\text{decision}(x) \rightarrow \text{references_continuity}(x) \vee \text{references_truth}(x)$
$\text{decision}(x) \nrightarrow \text{consequence}(x)$

26. Consequence

26.1 Definition

Consequence is the phase in which committed decision becomes applied effect.

Consequence is the most operationally forceful phase in the standing-state chain.

26.2 Semantic job

Consequence answers:

What effect now follows because a lawful decision stood?

26.3 What consequence is not

Consequence is not:

- truth
- continuity
- decision itself
- retroactive proof that all earlier phases were valid

Effect does not back-authorize upstream legitimacy.

26.4 Minimal formal boundary

consequence(x) → decision(x)
consequence(x) → applied_effect(x)
consequence(x) ↛ proof_of_upstream_validity(x)

27. Standing-State Regime Summary

The Standing-State Regime defines how the protocol moves from standing truth to preserved continuity, committed decision, and applied consequence.

Its canonical chain is:

truth → continuity → decision → consequence

Within this chain:

- truth forms standing claim,
- continuity preserves it,
- decision commits direction,

- consequence applies effect.

These phases are not interchangeable.

Each performs distinct semantic work and each must remain visible.

---

28. Anti-Collapse Summary of Both Regimes

A conforming system must preserve the following distinctions:

- signal is not acceptance
- acceptance is not scope
- scope is not presence
- presence is not threshold
- threshold is not truth
- truth is not continuity
- continuity is not decision
- decision is not consequence

These distinctions are the minimum conditions for non-collapsed protocol reasoning.

Minimal formal recap

accepted(x) → signal(x)
scoped(x) → accepted(x)
present(x) → scoped(x)
threshold_met(x) → present(x)

truth(x) → threshold_met(x)
continuity(x) → truth(x) ∨ references_truth(x)
decision(x) → continuity(x) ∨ references_continuity(x)
consequence(x) → decision(x)

29. Canonical Conclusion

Preparation makes standing possible.

Standing makes consequence lawful.

The protocol remains clean only when neither regime is allowed to impersonate the other.

3 - Containment, Resolution, and Lineage

30. Containment

30.1 Purpose

Containment governs whether an otherwise eligible irreversible transition may proceed.

Containment exists because:

- semantic readiness is not the same as permissibility,
- eligibility is not the same as authorization to proceed,
- and irreversible exit must sometimes be lawfully delayed, blocked, or held without corrupting phase meaning.

Containment therefore protects the protocol from mistaking:

- readiness for completion,
- delay for invalidation,
- or pause for lawful resolution.

30.2 HOLD

HOLD is the canonical containment expression.

HOLD is:

- cross-phase,
- orthogonal to the canonical semantic chain,
- non-standing in semantic kind,
- and non-resolutive in itself.

HOLD is not:

- a canonical phase,
- a truth state,
- a continuity state,
- a decision state,
- a consequence state,
- or a hidden fourth resolution outcome.

30.3 Semantic job

Containment answers:

May this matter proceed through an irreversible transition now?

That is its only job.

Containment does not answer:

- what the matter means,
- what stands,
- what is true,
- or what lawful resolution finally occurred.

30.4 Why containment must remain orthogonal

If containment becomes a phase, the architecture collapses.

The protocol must preserve this distinction:

- a **phase** answers what semantic state the matter is in
- **containment** answers whether movement into or through an irreversible step is permitted

These are different categories.

30.5 Effect of HOLD

A matter may be:

- semantically ready,
- threshold-eligible,
- properly bounded,
- and still not permitted to proceed because HOLD stands.

This is not contradiction. It is lawful containment.

30.6 Release

Release is the lifting of containment such that an otherwise eligible transition may proceed.

Release does not itself create:

- truth,
- continuity,
- decision,
- consequence,
- or resolution.

Release restores permissibility only.

30.7 What containment must not become

Containment must not become:

- indefinite silent burial,
- pseudo-resolution,
- hidden invalidation,
- or a loophole for avoiding visible state treatment.

If a matter remains held forever without visible later treatment, the system begins to corrupt its own architecture.

30.8 Minimal formal boundary

held(x) → ¬resolvable(x)
held(x) → ¬finalizable(x)
truth_eligible(x) ∧ held(x) → ¬truth_transition_permitted(x)
released(x) ∧ truth_eligible(x) → resolvable(x)
held(x) ↛ phase(x)

30.9 Containment summary

Containment governs irreversible permissibility without becoming part of the semantic chain.

HOLD may prevent transition - it does not create standing state.

Release may restore permissibility - it does not create standing state.

That distinction is non-negotiable.

---

31. Resolution

31.1 Purpose

Resolution defines the lawful ways standing matters may close, be negated, or be succeeded without semantic corruption.

Resolution exists because once a matter stands, later change must not occur through:

- disappearance,
- silent overwrite,
- hidden replacement,
- or untyped mutation.

Resolution is the protocol's explicit answer to:

what has now happened to this standing matter in lawful terms?

31.2 Closed resolution family

IAMMAI recognizes only three lawful resolution forms:

- **Finalize**
- **Invalidate**
- **Evolve**

No fourth resolution outcome is permitted by protocol.

That means:

- pause is not resolution,
- disappearance is not resolution,
- generic "update" is not resolution,
- and silent overwrite is not resolution.

---

32. Finalize

32.1 Definition

Finalize is the resolution act by which a standing matter is confirmed as closed in its present form.

Finalize means:

this matter stands as complete in its current protocol form.

32.2 Semantic job

Finalize answers:

Has this matter reached closure in the form in which it currently stands?

32.3 What Finalize is not

Finalize is not:

- threshold,
- temporary pause,
- invisible freeze,
- or successor creation.

A finalized matter may still later be referenced, but it is closed in its current form.

32.4 Minimal formal boundary

finalize(x) → standing(x)
finalize(x) → closed(x)
finalize(x) → resolvable(x)

33. Invalidate

33.1 Definition

Invalidate is the resolution act by which standing force is removed from a matter without erasing the fact that it stood.

Invalidate means:

this matter no longer stands, but its historical existence remains part of protocol memory.

33.2 Semantic job

Invalidate answers:

Has standing force been lawfully removed from this matter while preserving trace?

33.3 What Invalidate is not

Invalidate is not:

- deletion,
- disappearance,
- overwrite,
- or successor creation.

It is explicit negation without erasure.

33.4 Minimal formal boundary

$\text{invalidate}(x) \rightarrow \neg\text{standing}(x) \wedge \text{trace}(x)$
$\text{invalidate}(x) \rightarrow \text{resolved}(x)$
$\text{invalidate}(x) \nrightarrow \text{erase}(x)$

34. Evolve

34.1 Definition

Evolve is the resolution act by which a lawful successor state is created in explicit relation to a prior standing state.

Evolve means:

this matter continues through successor formation rather than overwrite.

34.2 Semantic job

Evolve answers:

What successor now stands in explicit continuity with what stood before?

34.3 Why Evolve must remain distinct

Evolve must remain distinct from Finalize and Invalidate because some matters are:

- not wrong,
- not to be erased,
- not merely closed,

  but must be lawfully succeeded.

Without Evolve, systems are forced into false choices:

- either invalidate what was once valid,
- or finalize isolated states without successor continuity.

Both are distortions when lawful succession is what actually occurred.

34.4 Minimal formal boundary

state_{n+1} = evolve(state_n, Δ)
ref(state_{n+1}) = state_n
evolve(state_n, Δ) ≠ overwrite(state_n)

35. Resolution family integrity

35.1 Closure law

The resolution family is closed.

resolution(x) ∈ {FINALIZE, INVALIDATE, EVOLVE}
¬∃ r resolution(r) ∉ {FINALIZE, INVALIDATE, EVOLVE}

35.2 Why closure matters

Without family closure, systems smuggle in pseudo-outcomes such as:

- unresolved stasis treated as result,
- hidden overwrite treated as correction,

- half-negation without trace,
- or implicit successor logic without explicit evolve relation.

IAMMAI forbids these.

35.3 Resolution summary

Resolution is the lawful way standing matters:

- close,
- lose standing,
- or continue through successor form.

It is not optional visibility.

It is the typed grammar of historical change.

---

36. Lineage

36.1 Purpose

Lineage preserves traceable identity across lawful change.

Without lineage:

- correction becomes erasure,
- replacement becomes disguise,
- and historical memory becomes false.

Lineage exists so that a system can change without pretending it never changed.

36.2 Definition

Lineage is the rule-set by which standing states remain historically connected through explicit predecessor-successor relation.

Lineage is not:

- surface similarity,
- narrative continuity,
- emotional familiarity,
- or mere repetition.

Lineage is preserved by lawful reference.

36.3 Semantic job

Lineage answers:

How does this matter remain historically traceable through lawful change?

36.4 Core lineage laws

36.4.1 No silent reset

A valid successor may not appear without explicit predecessor relation.

36.4.2 No overwrite

Correction must not replace prior standing state in place.

36.4.3 No orphan successor

A valid successor must carry predecessor reference.

36.4.4 Correction enters as succession

Where standing state changes lawfully, the new state must enter as successor rather than hidden edit.

36.5 Minimal formal boundary

valid(state_{n+1}) → ref(state_{n+1}) = state_n
¬∃ valid(state_{n+1}) such that predecessor_ref(state_{n+1}) = ∅
correction(state_n) = evolve(state_n, Δ)
continuity(state_{n+1}) → traceable_to(state_{n+1}, state_n)

36.6 Continuity and lineage

Protocol continuity is not raw persistence.

It is lineage-bearing preserved standing state.

A matter may move sharply and still preserve lineage.

Conversely, a matter may look stable while silently overwriting itself.

Therefore:

continuity is preserved by lawful successor reference, not by surface smoothness.

36.7 Lineage and resolution

Resolution depends on lineage to remain truthful:

- Finalize closes a state without successor requirement.
- Invalidate removes standing without erasing trace.
- Evolve creates successor relation explicitly.

Without lineage:

- invalidation becomes deletion,
- evolve becomes overwrite,
- finalization becomes ambiguous fossilization.

36.8 Common failure modes prevented by lineage

- hidden replacement
- historical laundering
- successor ambiguity
- correction erasure
- in-place mutation of standing state

36.9 Lineage summary

Lineage protects the protocol's historical truthfulness.

It ensures that:

- prior standing states remain part of history,
- successors remain explicit,
- and change remains visible without collapse into rewrite.

---

37. Integrated summary of containment, resolution, and lineage

Containment governs whether irreversible exit may proceed.

Resolution governs what lawful change to standing state has occurred.

Lineage preserves the historical relation among those changes.

These three must remain distinct:

- HOLD is not resolution
- resolution is not overwrite
- successor is not replacement-in-place

Minimal recap

held(x) → ¬resolvable(x)
finalize(x) → closed(x)
invalidate(x) → ¬standing(x) ∧ trace(x)
state_{n+1} = evolve(state_n, Δ)
ref(state_{n+1}) = state_n

38. Canonical Conclusion

Containment prevents premature transition.

Resolution types lawful change.

Lineage preserves history through that change.

Without these three, the protocol would still describe standing state, but it could not preserve it truthfully across time.

4 - Governance, Contribution, and Representation

39. Governance

39.1 Purpose

Governance defines who may lawfully perform protocol-significant acts, under what legitimacy basis, in relation to what matter, and with what attributable force.

Governance exists because semantic order alone does not answer:

- who may admit,
- who may finalize,
- who may invalidate,
- who may evolve,
- who may hold,
- who may release,
- who may commit,
- or who may apply consequence.

Without governance:

- access can impersonate authority,

- visibility can impersonate legitimacy,
- action can occur without attributable basis,
- and later audit cannot distinguish lawful change from ambient mutation.

39.2 Definition

Governance is the bounded authority grammar by which protocol-significant acts become:

- attributable,
- legitimacy-bounded,
- matter-scoped,
- and reviewable.

Governance is not:

- truth,
- authorship,
- contribution source,
- narrative explanation,
- or private certainty.

It answers only:

Who may lawfully do what, under what basis, in relation to which matter?

39.3 Core governance requirements

A conforming governance layer must preserve:

39.3.1 Authority class

What type of protocol-significant act the actor may lawfully perform.

39.3.2 Legitimacy basis

Why the act may stand.

39.3.3 Matter relation

What matter, state, or transition the act applies to.

39.3.4 Attribution

Who acted, or through what attributable system relation the act occurred.

39.3.5 Scope fit

Authority must remain bounded to the matter and act type for which it stands.

39.4 Authority classes

The foundational layer does not require one universal social ontology of actors, but it does require that protocol-significant authority classes remain distinguishable in function.

A core-compatible implementation must preserve distinctions functionally equivalent to:

- intake authority,
- standing-formation authority,
- continuity-preservation authority,
- commitment authority,
- application authority,
- containment authority,
- and resolution authority.

The exact labels may vary by profile. The functional distinctions may not silently collapse.

39.5 Legitimacy

Legitimacy is the basis by which an act may stand.

Legitimacy may arise from:

- role assignment,

- lawful delegation,
- procedural completion,
- institutional basis,
- jurisdictional basis,
- or other explicit admissible source.

A governance act without visible legitimacy basis is architecturally weak and often non-compliant.

39.6 Attributable governance

A protocol-significant act must not appear as though it happened by anonymous force.

Where a matter is:

- held,
- released,
- finalized,
- invalidated,
- evolved,
- committed,
- or applied,

the system must be able to preserve:

- who acted,
- what authority class was exercised,
- on what matter,
- under what legitimacy basis,

- and when.

39.7 Delegation

Delegation allows one actor or system to act under authority sourced from another lawful basis.

Delegation must not erase attribution.

A delegated act must preserve:

- delegated actor,
- delegating source,
- act type,
- matter relation,
- legitimacy basis,
- and time of action.

Delegation extends lawful action. It does not erase the chain of responsibility.

39.8 Governance is not semantic source

This distinction is mandatory.

Authority to act is not the same as **semantic source**.

A system may lawfully:

- authorize,
- finalize,
- invalidate,
- evolve,
- render,

- or transmit

material that it did not originate.

Governance must therefore never be treated as default authorship.

39.9 Minimal formal boundary

$\text{authorized}(a, act, m) \rightarrow \text{legitimate}(a, act, m)$
$\neg\text{authorized}(a, act, m) \rightarrow \neg\text{valid_governance_act}(a, act, m)$
$\text{delegated}(a2, a1, act) \rightarrow \text{traceable}(a2, a1, act)$
$\text{authorized}(a, act, m1) \wedge (m2 \neq m1) \rightarrow \neg\text{authorized}(a, act, m2)$
$\text{authorized}(a, act, x) \not\rightarrow \text{semantic_source}(a, x)$

39.10 Governance summary

Governance ensures that protocol-significant action remains:

- lawful,
- bounded,
- attributable,
- and non-anonymous.

It prevents the collapse by which:

the one who acted is mistaken for the one from whom the material came.

---

40. Contribution

40.1 Purpose

Contribution defines how semantic participation in protocol-relevant material remains distinguishable where multiple actors, systems, or layers participate in producing, shaping, authorizing, rendering, or carrying it.

Contribution exists because governance alone is not enough.

A system may correctly preserve:

- who had lawful authority to act,

while still failing to preserve:

- where the governing signal arose,
- who made it explicit,
- who structured it,
- who allowed it to stand,
- who made it legible,
- and who carried it downstream.

If those roles collapse, protocol history becomes semantically false even when governance attribution remains present.

40.2 Definition

Contribution is the grammar by which materially distinct semantic roles remain distinguishable across co-produced or co-carried material.

It is not:

- a status ranking,
- a moral score,
- a personality theory,
- or a generalized philosophy of collaboration.

It is an anti-collapse layer.

40.3 Contribution-role principle

A conforming system must preserve distinction among:

- **origin**
- **elicitation**
- **formalization**
- **authorization**
- **rendering**
- **transmission**

where those roles are materially distinct.

This principle does not require all six roles in every case.

It requires only that where distinct roles exist, they do not silently merge.

---

41. Origin

41.1 Definition

Origin is the role by which the governing signal, claim, insight, pattern, or semantic impulse first arose.

Origin answers:

Where did the governing semantic source first arise?

41.2 What origin does not imply

Origin does not by itself imply:

- admissibility,
- formal structure,
- lawful standing,
- public legibility,

- downstream transmission,
- or governance authority.

A thing may originate before it is:

- elicited,
- formalized,
- authorized,
- rendered,
- or transmitted.

---

42. Elicitation

42.1 Definition

Elicitation is the role by which latent, implicit, partial, or unstructured origin is drawn into explicit form.

Elicitation answers:

Who or what pulled the material into expressible visibility?

42.2 Distinction from origin

A system may elicit what it did not originate.

Elicitation is not default origination.

---

43. Formalization

43.1 Definition

Formalization is the role by which explicit material is converted into stable, bounded, structured, and referenceable form.

Formalization answers:

Who or what converted the material into durable semantic structure?

43.2 Formalization may include

- protocol wording,
- schema construction,
- doctrinal compression,
- structural decomposition,
- stable phrasing,
- semantic bounding,
- and record shaping.

43.3 Distinction from origin

Formalization is not origin.

A system may formalize what it did not originate.

---

44. Authorization

44.1 Definition

Authorization is the role by which material is allowed to stand, be used, be relied upon, or be published within lawful bounds.

Authorization answers:

Who or what allowed this material to enter standing, use, or release?

44.2 Distinction from source

Authorization does not imply semantic source.

A system may authorize material that it did not:

- originate,
- elicit,
- formalize,
- render,
- or transmit.

---

45. Rendering

45.1 Definition

Rendering is the role by which material is transformed into audience-legible, surface-legible, or domain-legible form without thereby changing its governing semantic source.

Rendering answers:

Who or what made this materially legible to the intended audience or surface?

45.2 Rendering may include

- explanatory phrasing,
- rhetorical shaping,
- interface presentation,
- visual translation,
- public-facing narration,
- or other forms of legibility work.

45.3 Distinction from authorship

Rendering must not be treated as authorship by default.

A system may render brilliantly and still not be the origin or primary semantic source.

---

46. Transmission

46.1 Definition

Transmission is the role by which already-authored, already-shaped, or already-authorized material is carried into downstream publication, delivery, propagation, issuance, or handoff.

Transmission answers:

Who or what carried this material downstream?

46.2 Distinction from creation

Transmission is not creation.

A system may carry material without having:

- originated it,
- formalized it,
- or rendered it.

---

47. Non-impersonation law of contribution roles

No contribution role may silently impersonate another.

Accordingly:

- origin must not be overwritten by formalization,
- elicitation must not be treated as origination merely because it made the material explicit,
- formalization must not be confused with origin,

- authorization must not be mistaken for semantic source,
- rendering must not be treated as authorship by default,
- transmission must not be treated as creation merely because it carried the material downstream.

This law exists to prevent authorship-collapse in:

- human systems,
- machine systems,
- and hybrid systems.

47.1 Minimal formal boundary

origin(a, x) = formalization(a, x) only_if explicitly_recorded(a, x)
elicitation(a, x) $\nrightarrow$ origin(a, x)
authorization(a, x) $\nrightarrow$ semantic_source(a, x)
rendering(a, x) $\nrightarrow$ authorship(a, x)
transmission(a, x) $\nrightarrow$ creation(a, x)
materially_distinct_roles(r1, r2, x) → r1 ≠ r2

47.2 Contribution summary

Contribution preserves semantic fairness and protocol truthfulness where multiple actors or systems participate in the same material.

It prevents the collapse by which:

touching the material later is mistaken for being where the material came from.

---

48. Representation

48.1 Purpose

Representation ensures that the distinctions protected by the protocol remain preservable in implementation.

The protocol does not stop at prose doctrine.

If implementation flattens those distinctions into opaque storage, the protocol becomes operationally false even where its language appears correct.

Representation therefore preserves enough structure that a later reader, auditor, or system can reconstruct:

- what matter existed,
- what phase/state relation applied,
- what governance act occurred,
- what resolution occurred,
- what predecessor/successor relation held,
- and what contribution roles were materially distinct.

48.2 Definition

Representation is the minimal object and record grammar by which protocol-relevant material remains:

- referenceable,
- distinguishable,
- reconstructable,
- and historically coherent.

Representation is not:

- one mandatory database schema,
- a product-specific storage design,
- a UI contract,
- or a transport layer.

It is the minimum semantic obligation of implementation.

48.3 Representation principle

A conforming implementation must preserve semantic distinctions at the object/record level sufficient to prevent:

- phase collapse,
- lineage loss,
- opaque governance,
- silent overwrite,
- and contribution-role collapse.

48.4 Minimal representational requirements

Where applicable, a conforming implementation should be able to preserve:

- matter references,
- phase/state relation,
- transition references,
- predecessor/successor references,
- governance action references,
- legitimacy references,
- containment references,
- resolution references,
- contribution-role references,
- and temporal order.

---

49. Governance action record

A governance-relevant act should remain visible in a shape conceptually equivalent to:

```
governance_action_record = {
  governance_action_id,
  matter_ref,
  action_type,
  authority_class,
  actor_ref?,
  legitimacy_basis_ref?,
  delegated_from_ref?,
  target_transition_ref?,
  target_state_ref?,
  contribution_role_map_ref?,
  acted_at
}
```

This is illustrative, not implementation-binding syntax.

Its purpose is to show that governance must remain attributable, bounded, and recoverable.

---

50. Contribution-role record support

Where materially distinct contribution roles exist, the implementation should be capable of preserving an explicit contribution-role relation or equivalent structure.

A foundationally compatible shape may be:

```
contribution_role_record = {
  contribution_role_id,
  matter_ref,
  artifact_ref?,
  role_type, // ORIGIN | ELICITATION | FORMALIZATION | AUTHORIZATION | RENDERING |
TRANSMISSION
  actor_ref?,
  related_governance_action_ref?,
  basis_ref?,
  recorded_at?
}
```

This does not impose one universal authorship ontology.

It preserves enough structure that distinction is not lost.

---

51. Resolution record support

Resolution should remain visible as typed change rather than generic mutation.

A foundationally compatible shape may be:

```
resolution_record = {
  resolution_id,
  target_state_ref,
  resolution_type, // FINALIZE | INVALIDATE | EVOLVE
  predecessor_ref?,
  legitimacy_basis_ref?,
  acted_at
}
```

This preserves:

- what resolution occurred,
- to what state,
- under what basis,
- and with what predecessor relation where relevant.

---

52. Representation anti-collapse rules

A conforming representation layer must not:

52.1 Flatten phases

Preparation and standing-state distinctions must remain recoverable.

52.2 Flatten resolution

Finalize, Invalidate, and Evolve must not be silently merged into generic “updated” or “closed” states.

52.3 Flatten governance

Protocol-significant action must not become opaque mutation.

52.4 Flatten contribution

Materially distinct contribution roles must not collapse into one undifferentiated authorship field.

52.5 Flatten lineage

Successor relation must not disappear into in-place replacement.

---

53. Canonical Conclusion

Governance defines who may act.

Contribution defines who contributed what kind of semantic role.

Representation ensures those distinctions can survive implementation.

Without these three, the protocol could still describe lawful standing, but not preserve truthful authorship, lawful action, or reconstructable history.

5 - Conformance, Integrated Core Summary, and Final Canonical Close

54. Conformance

54.1 Purpose

Conformance defines what must remain true if a later implementation, profile, or adaptation is still to count as IAMMAI-compatible.

The protocol is not protected merely because a later system claims inspiration from it.

It is protected only when the implementation preserves the foundational semantics that give IAMMAI its identity.

Conformance therefore exists to distinguish:

- legitimate variation

  from

- semantic drift,

and:

- implementation freedom

  from

- collapse of the constitutional core.

54.2 Definition

Conformance is the rule-set by which implementations are evaluated for fidelity to:

- canonical phase order,
- preparation/standing distinction,
- containment semantics,
- resolution family closure,
- lineage preservation,
- governance attribution,
- contribution-role distinction,
- and representational recoverability.

Conformance is not:

- stylistic uniformity,
- one technical stack,
- or one naming system.

Variation is allowed. Collapse is not.

54.3 Foundational conformance principle

A later implementation may vary in:

- naming,
- storage design,
- transport,
- interface,
- workflow orchestration,
- domain-specific criteria,
- and technical realization

only if it preserves the core semantic identity of IAMMAI.

---

55. Core Conformance Invariants

A conforming implementation must preserve the following invariants.

55.1 Phase-order invariant

Preparation must precede standing-state formation.

$\text{truth}(x) \rightarrow \text{threshold_met}(x)$
$\text{decision}(x) \rightarrow \text{truth}(x) \lor \text{references_truth}(x)$
$\text{consequence}(x) \rightarrow \text{decision}(x)$

55.2 Preparation / standing distinction invariant

No preparation phase may silently claim standing force.

signal(x) $\nrightarrow$ standing(x)
accepted(x) $\nrightarrow$ standing(x)
scoped(x) $\nrightarrow$ standing(x)
present(x) $\nrightarrow$ standing(x)
threshold_met(x) $\nrightarrow$ standing(x)

55.3 Hold invariant

Containment must remain orthogonal to standing-state formation and lawful resolution.

held(x) $\rightarrow$ ¬resolvable(x)
held(x) $\rightarrow$ ¬finalizable(x)
held(x) $\nrightarrow$ resolution(x)

55.4 Resolution-family invariant

Resolution must remain closed to Finalize, Invalidate, and Evolve.

resolution(x) $\in$ {FINALIZE, INVALIDATE, EVOLVE}
$\neg\exists$ r resolution(r) $\notin$ {FINALIZE, INVALIDATE, EVOLVE}

55.5 Lineage invariant

Valid successor state must preserve predecessor relation.

valid(state_{n+1}) $\rightarrow$ predecessor_ref(state_{n+1}) $\neq \varnothing$
valid(state_{n+1}) $\rightarrow$ ref(state_{n+1}) = state_n

55.6 Governance-attribution invariant

A valid governance act must remain attributable and legitimacy-bounded.

valid_governance_act(a, act, m) $\rightarrow$ authorized(a, act, m) $\wedge$ legitimate(a, act, m)

55.7 Contribution-distinction invariant

Where materially distinct contribution roles exist, the implementation must preserve distinction sufficient to prevent silent authorship collapse.

materially_distinct_roles(x) → preserved_distinction(x)

55.8 Representation-recoverability invariant

Implementation must preserve enough structure that protocol-relevant distinctions can be reconstructed later.

valid_protocol_record(x) → recoverable_semantic_distinction(x)

56. Prohibited Patterns

The following patterns are non-conformant.

56.1 Phase collapse

Treating preparation and standing-state phases as though their distinctions do not matter.

56.2 Dark standing

Allowing standing state to appear without lawful transition.

56.3 Dark consequence

Allowing consequence to appear as though it authorized itself.

56.4 Hidden fourth outcome

Creating pseudo-resolution outcomes outside Finalize, Invalidate, and Evolve.

56.5 Silent overwrite

Replacing standing state in place without visible successor relation.

56.6 Opaque governance act

Allowing protocol-significant action without attributable authority and legitimacy relation.

56.7 Contribution-role collapse

Silently treating origin, elicitation, formalization, authorization, rendering, or transmission as the same role where they are materially distinct.

56.8 Representation flattening

Encoding records such that required distinctions are no longer recoverable.

56.9 Retroactive proof collapse

Treating downstream consequence as proof that upstream legitimacy or truth must have existed.

Minimal formal non-conformance boundary

phase_collapse(x) → nonconformant(x)
dark_standing(x) → nonconformant(x)
dark_consequence(x) → nonconformant(x)
silent_overwrite(x) → nonconformant(x)
collapsed_resolution_family(x) → nonconformant(x)
opaque_governance_act(x) → nonconformant(x)
collapsed_contribution_roles(x) → nonconformant(x)
representation_flattening(x) → nonconformant(x)
retroactive_proof_collapse(x) → nonconformant(x)

57. Permitted Variation

The following may vary across implementations, provided foundational semantics remain intact:

- terminology,
- object naming,
- storage layout,
- transport mechanisms,
- interface design,
- automation style,
- institutional embedding,

- and domain-specific criteria.

The following may **not** vary without breaking compatibility:

- canonical phase order,
- distinction between preparation and standing state,
- orthogonality of HOLD,
- closure of the resolution family,
- lineage-preserving successor logic,
- attributable governance,
- contribution-role non-collapse where materially relevant,
- and recoverable semantic representation.

---

58. Minimal Conformance Questions

A serious reader, implementer, or auditor should be able to ask:

58.1 Canonical architecture

Can this system distinguish preparation from standing-state formation?

58.2 Containment

Can it show when a matter was held, released, or otherwise blocked from irreversible progression?

58.3 Resolution

Can it distinguish Finalize, Invalidate, and Evolve?

58.4 Lineage

Can it show predecessor/successor relation where lawful succession occurred?

58.5 Governance

Can it show who acted, under what authority, on what basis, in relation to what matter?

58.6 Contribution

Where material was co-produced, can it show who originated it, who elicited it, who formalized it, who authorized it, who rendered it legible, and who carried it downstream?

58.7 Representation

Can the implementation recover the distinctions the doctrine requires?

If the answer to these questions is systematically no, the implementation is not IAMMAI-compatible in core semantic terms.

---

59. Integrated Core Summary

IAMMAI is a public constitutional protocol for keeping candidate material, standing state, lineage, authority, contribution, and consequence in lawful order without semantic collapse.

Its canonical semantic chain is:

signal → acceptance → scope → presence → threshold → truth → continuity → decision → consequence

Within this architecture:

- **Preparation Regime** qualifies candidate material without creating standing state,
- **Standing-State Regime** forms, preserves, commits, and applies standing state,
- **Containment** governs irreversible permissibility without becoming a semantic phase,
- **Resolution** is lawfully limited to Finalize, Invalidate, and Evolve,
- **Lineage** preserves traceable successor relation and forbids silent overwrite,
- **Governance** binds protocol-significant action to lawful authority and legitimacy,
- **Contribution** prevents semantic-role collapse across co-produced material,

- **Representation** preserves distinctions in implementable form, and
- **Conformance** protects the protocol from drift across later adaptations.

The protocol ensures that what is available, what may count, what stands, what changes, and what follows remain clear, bounded, and historically truthful.

---

60. Canonical Close

IAMMAI exists so that systems may proceed lawfully.

ORIGINATING AUTHOR AND PUBLIC RELEASE AUTHORITY

Marko Markota

EDITION

Public Constitutional Protocol

HISTORICAL LINEAGE NOTE

Selected preconstitutional lineage references are preserved separately as non-normative historical material and do not supersede this constitutional core.

CONSTITUTIONAL STATUS

This document is the present human-readable constitutional core. Any machine-readable projection, companion note, implementation artifact, or historical lineage material remains subordinate to this text unless explicitly superseded by a later authorized edition.